CypressWatch — Final SwampHacks XII Plan (All Sponsors)

Oct 16, 2026 · @Satyabrata Das

Overview

CypressWatch is our PlanIT Control platform for Sperry Tech: it watches the environment around 50+ real power plant sites, warns crews the moment conditions turn dangerous, scores every site with an ML hazard index (0–10), and sends a next-day safety briefing at 6 PM.

Our goal is to win, so every sponsor challenge that fits is a real, working feature with one clear job — no logos, no token integrations. We are entering 13 challenges with one coherent product; hardware is last priority and is decided at hour 16.

> **One-line pitch:** "CypressWatch turns scattered weather, lightning, flood, fire, and wildlife data into one risk score per job site, stops work the moment lightning gets close, and keeps a tamper-proof safety record every company on the project can verify."

The rule that protects us: every feature reaches a simple working version before anything gets polished, and we freeze features at hour 30.

What Sperry's reps told us

We asked the Sperry reps four questions; their answers set our priorities.

Question	Their answer	What we change
Which hazards matter most?	Depends on geography; for Florida, lightning	The lightning stop-work countdown is the centerpiece: first in the demo, most polished, first on the dashboard

Question	Their answer	What we change
Who uses it day to day?	A mix, primarily supervisors	The supervisor map and site page are our main screens; the crew view supports them
How do crews get hazard information today?	No one definitive source; it breaks down during something we didn't catch (ask again)	Our pitch opens with it: "There's no definitive source. CypressWatch is that source."
Do crews work with poor signal?	Yes, very much	The crew app is offline-first (below)

Offline-first crew app

- The 30-minute countdown runs on the device from the last strike time, so it keeps counting if the connection drops.
- The latest briefing, site risk, and hazard map are saved on the phone.
- Crew hazard reports queue offline and upload automatically when signal returns.
- Every screen shows "last updated X minutes ago" so old data is never mistaken for live data.

Sperry PlanIT Control: requirements and how we meet them

Sperry's brief lists six required features; CypressWatch covers all six and adds three differentiators on top.

Sperry requirement	How CypressWatch delivers it
One combined risk picture per site, at least 50 real power plants (weather, heat, lightning, flooding, wildfire, soil saturation, wildlife)	50+ Florida plants from EIA-860M, each with a site page combining every hazard into one breakdown
Instant alerts: lightning within 10 miles → stop-work with a 30-minute all-clear countdown that restarts on each new strike	Lightning engine checks every new flash against each site's 10-mile radius; countdown restarts on each flash and syncs live to crew phones and the Site Beacon

Sperry requirement	How CypressWatch delivers it
Instant alerts: river sites get a flood-risk warning after a night of heavy rain	Overnight rainfall total + river-site flag + USGS river gauges + NWS flood warnings → flood-risk alert at sunrise
Next-day safety briefing every evening at 6 PM local time	Scheduler on Vultr fires at 6 PM in each site's time zone; Gemini writes a grounded briefing in English and Spanish
Crew hazard reporting: photo, pin, share with team, build a database for future project owners	Snap a photo → Gemini suggests hazard type and severity → pinned on the map, shared live, stored in the hazard registry and anchored on Solana
Supervisor dashboard: map ranking sites by risk	Map of all sites colored by 0–10 score, a ranked list, and filters by hazard
Hazard index 0–10 from an ML model trained on historical data	Gradient-boosted model trained on NOAA Storm Events and OSHA injury records with historical weather, evaluated on held-out recent years

Our differentiators

- **Best work window:** the 6 PM briefing recommends tomorrow's safest working hours per site (e.g., start at 6 AM, stop before the heat index passes 105°F and storms arrive).
- **"Why is this site an 8?"** Gemini explains each score in plain language from the model's top contributing factors.
- **Works without signal:** the crew app keeps the countdown, briefing, and map available offline and uploads reports when signal returns.
- **Verifiable safety record:** stop-work events and hazard reports are anchored on Solana, so any company on the project can prove the record was never edited.

All 13 challenges we enter

Every challenge gets one clear job in the product, so no two sponsors overlap and every integration is visible in the demo.

#	Challenge	Prize	CypressWatch's unique use	Owner
1	**PlanIT Control — Sperry Tech**	3 winners: iPad Air, AirPods, gift cards + internship interviews	The whole platform: 50+ sites, lightning countdown, flood alerts, 6 PM briefing, crew reports, ranked map, ML index	Everyone (M4 leads)
2	**Best Overall (Vobile)**	$2,500, 3 winners	Complete, real-world product with deep engineering	Everyone
3	**Best Use of AI/ML**	Brick	Custom hazard-index model trained on real incident data, with evals, explanations, and a public repo + README	M3
4	**Best Use of Tiger Data**	Stream Mini Decks	Time-series core: conditions, forecast snapshots, lightning flashes, and scores in hypertables; continuous aggregates; compression; forecast drift	M4
5	**Best Use of Gemini API**	MLH swag	Grounded bilingual 6 PM briefings, multimodal hazard-photo triage, and plain-language score explanations	M3

#	Challenge	Prize	CypressWatch's unique use	Owner
6	**Best Use of Vultr**	M5Stack Tab5	Runs ingestion workers, the lightning engine, the 6 PM scheduler, and model serving; Object Storage for photos and model files	M4
7	**Best Use of Auth0**	Wireless headphones	Supervisor / crew / project-owner roles, passwordless crew login, step-up MFA to override a stop-work, scoped access for the AI briefing agent	M1
8	**Best Use of DeepSpace**	$250 + credits, 2 winners	Real-time sync: stop-work alerts, the countdown, and queued crew reports reach every device	M2
9	**Best Use of Solana**	Assorted prizes	Safety Ledger: per-site on-chain Merkle roots for stop-work events and hazard reports, verifiable by any future project owner	M3
10	**Easy On The Eyes**	Acer monitor	Ranked risk map, site risk breakdown, and a glove-friendly, high-contrast, offline-ready crew view	M1 + M2
11	**Best Domain Name (GoDaddy Registry)**	Digital gift card	Register a memorable domain in hour one (check which endings qualify)	M1

#	Challenge	Prize	CypressWatch's unique use	Owner
12	**Best First Timers**	Logitech bundle	Eligible only if at least 2 of our 4 are at their first hackathon	—
13	**Best Hardware Hack**	Raspberry Pi 4 Kit	**Last priority, decided at hour 16.** Site Beacon at the trailer (see the last section)	M2, or whoever is ahead

Not entering, and why

These four can't be done properly inside CypressWatch, and a forced entry would look unfocused to judges:

- **Capital One Nessie:** judged on reimagining banking; a safety platform has no honest banking use.
- **Roblox Studio:** the project must be built inside Roblox Studio in Luau.
- **Entertainment & Creative Technology:** judged on games, media, and storytelling.
- **Least Vibe Coded:** judged on minimal AI-generated code, which conflicts with how we'll build at this speed.

Data and AI sponsors in detail

The model decides scores, deterministic rules decide alerts, and Gemini explains; no AI output ever triggers or cancels a stop-work.

Best Use of AI/ML: the hazard index model

- **Training data:** NOAA Storm Events (thunderstorm, lightning, flash flood, heat, wildfire events by place and date) and OSHA Severe Injury Reports (heat and weather injuries with locations), joined to historical daily weather for each area from Open-Meteo's archive.
- **Label:** did a hazardous event or injury occur near the site on that day.
- **Features:** max heat index, rainfall (24 h and 7-day), soil moisture, wind gusts, storm instability, plus static site features (FEMA flood zone, distance to a river, plant type).

- **Model:** gradient-boosted classifier (e.g., LightGBM or XGBoost); predicted probability calibrated and scaled to 0–10.
- **Evaluation:** time-based split (train on older years, test on the two most recent), compared with a heat-index-only baseline; report AUC, calibration, and feature importance.
- **Explanations:** per-site top contributing factors (SHAP values) feed Gemini's "why is this site an 8?" text.
- **Fallback:** a rule-based 0–10 score ships first so the dashboard works while the model trains.
- **Track requirements:** public repo, README section on data, model, evals, and limitations.

Best Use of Tiger Data: the time-series core

- **Hypertables:** `conditions` (site, time, heat index, rain, soil moisture, wind), `forecasts` (site, issued_at, valid_at, values), `lightning_flashes` (time, lat, lon), `hazard_scores` (site, time, score, model version), `crew_reports`, `alerts`.
- **Continuous aggregates:** hourly and daily max heat index per site, overnight rainfall totals (feeds the flood rule), flashes within 10 miles per site per 5 minutes, daily peak score per site.
- **Forecast drift:** compare forecast snapshots over time, e.g. "the 3 PM storm chance for Site 12 rose from 20% to 60% since 7 AM."
- **Compression and retention** on raw lightning and condition data, with storage savings shown in the README.
- **One database for everything:** relational site and user data next to the time-series, and the same tables produce the ML training features.
- **Distance checks** in plain SQL with a haversine formula; use PostGIS only if the service has it enabled, with Turf.js as the app-side backup.

Best Use of Gemini API: three distinct jobs

1. **6 PM next-day briefing:** Gemini receives only the structured evidence (forecast, alerts, score, factors, crew reports) with source IDs and writes the briefing in English and Spanish. It must cite sources and say when data is missing; a validator rejects briefings that mention hazards not in the evidence.
2. **Hazard-photo triage:** a crew photo returns structured JSON (hazard type, severity, short description, confidence); the supervisor confirms before it becomes official.

3. **Score explanations:** turns the model's top factors into one plain sentence per site.

Evals (15–20 cases): no invented hazards, correct source citations, stale data flagged, forecasts not described as warnings, photo triage tested on sample images.

Platform sponsors in detail

Each platform sponsor owns one layer: Vultr runs it, Auth0 decides who can do what, DeepSpace makes it live, and Solana makes it provable. The optional Site Beacon is covered at the end of the doc.

Best Use of Vultr: where everything runs

- Compute instance(s) running the API, the ingestion workers (lightning about every minute, conditions every 15 minutes), the lightning and flood alert engines, the 6 PM per-time-zone scheduler, and the model inference service.
- Object Storage for crew hazard photos, trained model files, and archived briefings.
- Deploy in hours 0–4 and redeploy continuously; include a deployment diagram and live URL in Devpost.

Best Use of Auth0: who can do what

- **Three roles:** supervisor (all assigned sites), crew member (own site, reports, acknowledgments), project owner (read-only hazard history).
- **Passwordless login** for crews in the field.
- **Step-up MFA** before a supervisor overrides a stop-work or issues an early all-clear, so the most safety-critical action needs the strongest proof.
- **AI agent scoping:** the briefing agent calls our API with the supervisor's scoped token, so it can only read sites that supervisor manages. Check Auth0's AI-agent features (they mention securing AI apps) and use them if they fit.
- The server enforces every role check; hiding a button is not security.

Best Use of DeepSpace: what makes it live

- Real-time sync of stop-work alerts, the 30-minute countdown, all-clear events, crew acknowledgments, and the crew report feed between supervisor and crew devices.
- Demo: two phones side by side; a simulated strike starts the countdown on both instantly.

- Hour 0–2 check: confirm DeepSpace can trust our Auth0 identity, so we never add a second login.
- Devpost: list the DeepSpace features used, where judges can see them, and test access instructions with no secrets exposed.

Best Use of Solana: the Safety Ledger

Problem: a utility, its contractors, insurers, and future project owners all rely on a site's safety history. A record in one company's database can be edited after an incident, and the others must trust that company.

Solution:

1. Every stop-work event, all-clear, acknowledgment, and confirmed crew report is hashed.
2. Each hour, each site's hashes are combined into one Merkle root.
3. A small Anchor program stores roots in a per-site account (PDA) on devnet: `anchor_root(site_id, period, root, count)`. Fallback: the SPL Memo program if Anchor setup stalls.
4. A Verify page recomputes any record's hash and Merkle proof and checks it against the on-chain root.
5. Only hashes, counts, and timestamps go on-chain; no names, photos, or locations.

Why Solana: 50 sites × 24 hourly roots is 1,200 transactions a day, which is practical only because Solana's fees are near zero and confirmations are fast. Any company can verify a site's history without trusting another company's database.

Demo: verify a stop-work record (match) → edit one word in the database → verify again (mismatch).

Best Hardware Hack: Site Beacon (last priority)

Built only if the core works at the hour-16 checkpoint. The full plan and the parts list are in the last section of this doc.

Easy On The Eyes

- Supervisor map with sites colored by 0–10 score, a ranked list, and hazard filters.
- Site page with a hazard breakdown, forecast-drift chart, and score explanation.

- Crew view designed for the field: large countdown, big buttons, high contrast for bright sunlight, color always paired with icons and text.

Best Domain Name

Register in hour one through GoDaddy Registry and confirm which domain endings qualify for the prize.

Data sources per hazard

These are suggestions to verify in hours 0–2; Sperry allows any source. Every live source needs a demo injection button, because conditions may be calm when judges arrive.

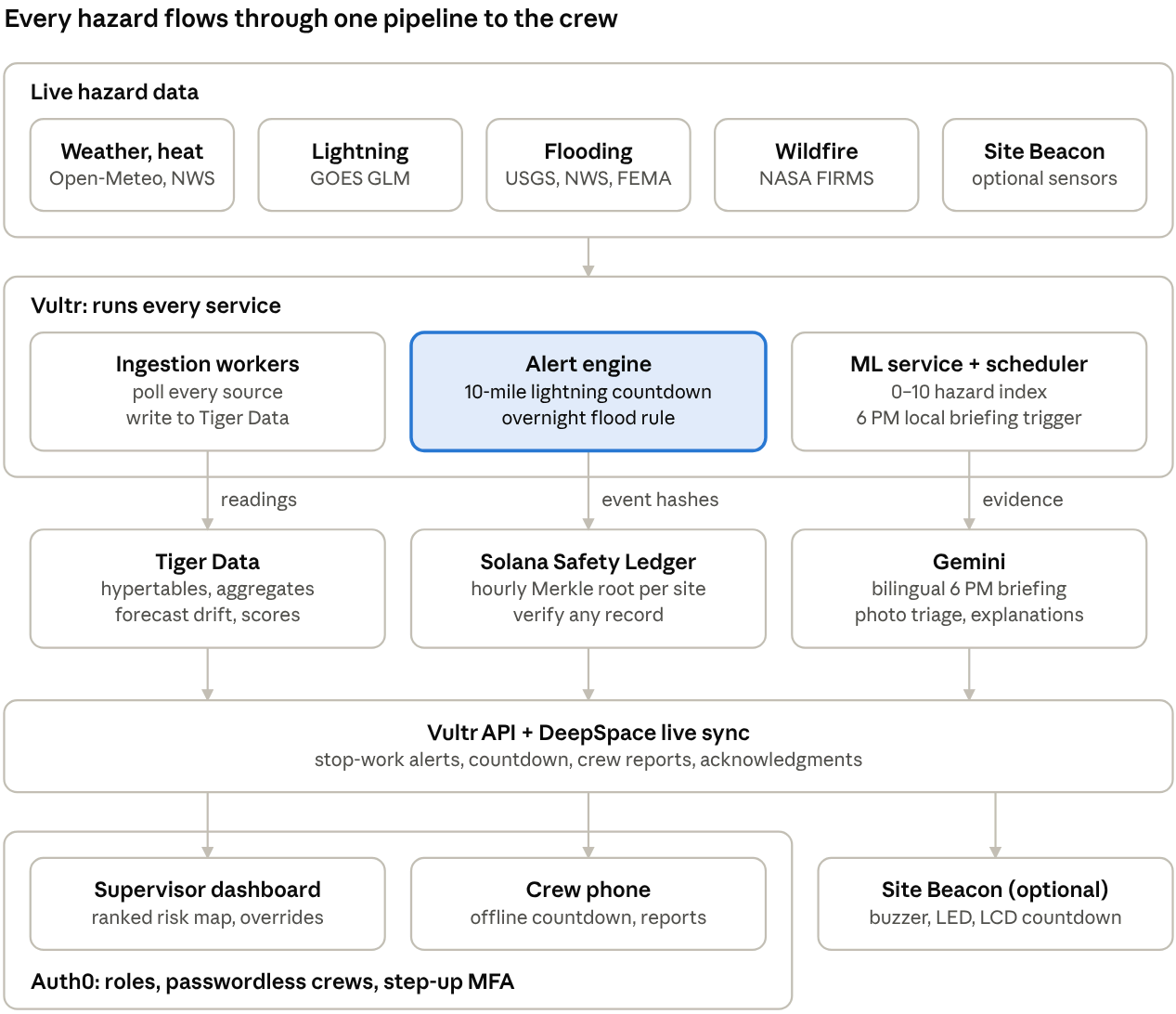
Hazard	Live data	Historical / static	Demo control
Sites	—	EIA-860M power plant list (filter to 50+ Florida plants)	—
Heat	Open-Meteo or NWS hourly forecast (apparent temperature / heat index)	Open-Meteo archive	Heat spike
Lightning	GOES satellite lightning mapper (GLM) flash data, free on AWS	NOAA Storm Events	**Simulate strike near a site**
Flooding	Overnight rainfall + USGS river gauges + NWS flood warnings; NOAA Tides for coastal sites	FEMA flood zone per site, National Levee Database	Heavy-rain night
Soil saturation	Open-Meteo soil moisture + 7-day rainfall	Open-Meteo archive	—
Wildfire	NASA FIRMS active fires (free key) + NWS Red Flag Warnings	NOAA Storm Events	Fire nearby
Wildlife	Crew reports; optionally iNaturalist observations nearby	Crew report history	Sample report

Hazard	Live data	Historical / static	Demo control
Injuries (labels)	—	OSHA Severe Injury Reports, Iowa Environmental Mesonet warning archive	—

Notes

- Satellite lightning data are flashes, not exact ground strikes; confirm with Sperry that this counts.
- Many NWS alerts (heat advisories, Red Flag Warnings) cover forecast zones rather than drawn polygons; fetch the zone shapes for those.
- Label demo injections clearly as simulated in the UI.

System architecture



CypressWatch architecture · data in, alerts out

Data moves top to bottom: Vultr workers pull every source into Tiger Data, the highlighted alert engine is the only path that can start or end a stop-work, and DeepSpace pushes each event to supervisors and crews at the same moment. Crew phones keep the countdown and latest data offline; the Site Beacon is optional.

Team roles

Two members build the frontend plus the light backend it needs; the other two own AI/ML and the backend platform. Each member owns their sponsors end to end, including the Devpost write-up and 30-second spotlight.

Member	Focus	Owns	Sponsors owned	First milestone (by hour 8)
M1 — Frontend: Supervisor side	Frontend + light backend	Supervisor map and ranked list, site risk page, score and forecast-drift charts, Solana Verify page UI, Auth0 login, roles, and step-up MFA (frontend + API middleware), domain	Easy On The Eyes, Auth0, Domain	Login working; 50 sites on the map colored by score
M2 — Frontend: Crew side	Frontend + light backend	Offline-first crew app (local countdown, saved briefing and map, queued reports), photo report flow and upload endpoint, DeepSpace live sync, Site Beacon if chosen at hour 16	DeepSpace, Hardware (last priority); shares Easy On The Eyes	Crew view showing a site's risk; countdown running locally
M3 — AI/ML Lead	AI + Solana	Historical data, hazard-index model, evals, SHAP explanations, Gemini briefing, photo triage, and translation, AI README; Solana Safety Ledger after the model ships (hour 16+)	AI/ML, Gemini, Solana	Training dataset built; rule-based 0–10 score in the API

Member	Focus	Owns	Sponsors owned	First milestone (by hour 8)
M4 — Backend / Platform Lead	Backend + infrastructure	EIA-860M sites, ingestion workers, Tiger Data schema, aggregates, and compression, lightning and flood engines, simulate buttons, 6 PM scheduler, API, Vultr deployment	Sperry core, Tiger Data, Vultr	Live conditions for all sites in Tiger Data, deployed on Vultr

Shared by everyone: Sperry (the whole product), Best Overall, documentation for their own components, and demo rehearsal.

How the frontend pair splits work: M1 owns everything a supervisor sees, M2 owns everything a crew member sees. They share one component library and color set so the app looks like one product.

Load check: M4 is busiest in hours 0–12. If M4 falls behind, M1 takes Vultr deployment (mostly configuration) and M3 takes the 6 PM scheduler.

36-hour build timeline

All four members work in parallel from hour 0. The core works end to end by hour 16, every sponsor feature has a simple working version by hour 24, and features freeze at hour 30.

Hours	M1 — Frontend: Supervisor	M2 — Frontend: Crew	M3 — AI/ML	M4 — Backend / Platform
0–2: Setup	Repo, app shell, shared components; Auth0 app; register domain	Crew app shell (installable web app); DeepSpace account; check DeepSpace works with Auth0	Follow-up questions to Sperry reps (with M4); start historical data downloads; Gemini key	50+ Florida plants from EIA-860M; Tiger Data service; Vultr server
2–8: Vertical slice	Login and roles; 50 sites on the map colored by score; ranked list	Crew view of one site; local countdown logic	Training dataset; rule-based 0–10 score endpoint	Conditions ingestion into hypertables; API; first Vultr deploy
8–16: Core features	Site risk page; step-up MFA on stop-work overrides	Offline caching; photo report flow and upload; DeepSpace live sync of alerts	Model v1 with time-based eval vs. baseline; Gemini photo triage	Lightning ingestion and 10-mile countdown engine; simulate-strike button; flood rule
Hour 16: Checkpoint	Does the core work end to end (ranked map, lightning countdown, score, crew reports)? If yes and someone is ahead, start the Site Beacon. If not, skip hardware.			

Hours	M1 — Frontend: Supervisor	M2 — Frontend: Crew	M3 — AI/ML	M4 — Backend / Platform
16–24: Every sponsor working	Score and forecast-drift charts; Solana Verify page	Queued-report sync; briefing screen; Site Beacon if chosen	Model replaces rules; SHAP explanations; Gemini 6 PM briefing (EN/ES); Solana hashing and Merkle roots on devnet	Continuous aggregates; compression; 6 PM scheduler per time zone; wildfire and soil layers; **check in with Sperry reps**
24–30: Prove quality	UI polish and accessibility	Airplane-mode tests; sunlight-readable mode	15–20 Gemini evals; AI README; Solana verify end to end	Stale data, source outages, duplicate flashes; server role checks; architecture diagram
30–34: Submission	Easy On The Eyes, Auth0, Domain write-ups; screenshots	DeepSpace (and Hardware) write-ups	AI/ML, Gemini, Solana write-ups	Sperry, Tiger Data, Vultr write-ups; backup demo video
34–36: Rehearsal	Everyone runs the full demo three times and practises their sponsor spotlights			

If something stalls for more than an hour: ship its simplest working version (e.g., Memo program instead of Anchor, rule-based score instead of the model) and move on. A working simple version scores; a broken ambitious one doesn't.

Expo demo

Judging is an expo: judges come to our table, and each sponsor's judges care mostly about their own technology. We need one 90-second core demo plus a 30-second spotlight per sponsor.

Core demo (90 seconds)

1. **The problem (10 s):** "Your reps told us there's no definitive source for hazard data and crews often have no signal. CypressWatch fixes both."
2. **Ranked map (15 s):** log in as a supervisor; 50+ real power plants ranked by 0–10 hazard score. Open the top site: "Site 12 is an 8 because of heat index and saturated soil."
3. **Lightning (25 s):** press "simulate strike near Site 12". The stop-work alert and 30-minute countdown appear on the crew phone. Simulate a second strike; the countdown restarts.
4. **No signal (20 s):** put the crew phone in airplane mode. The countdown keeps running. Photograph a hazard; the report queues. Turn signal back on: the report uploads, Gemini suggests its type and severity, and it appears on the supervisor map.
5. **Proof (20 s):** open the Verify page; the stop-work record matches the root on Solana. Edit one word in the database; verification fails.

30-second sponsor spotlights

Sponsor	What to show
Sperry	The six required features in order, plus tomorrow's 6 PM briefing with the recommended work window and the offline crew app
AI/ML	Model card: training data, time-based eval vs. baseline, calibration, SHAP explanation for one site
Tiger Data	Hypertables, a continuous aggregate powering the map, forecast drift query, compression savings
Gemini	Grounded bilingual briefing with source IDs, photo triage JSON, an eval that catches an invented hazard
Vultr	Running workers, scheduler, model service, and Object Storage; the live URL

Sponsor	What to show
Auth0	Crew vs. supervisor views, passwordless crew login, step-up MFA prompt on a stop-work override
DeepSpace	Two devices updating instantly; queued reports syncing after airplane mode; which DeepSpace features we use and where
Solana	Anchor program, per-site accounts, Merkle proof verification, cost of 1,200 roots a day
Easy On The Eyes	Map, site page, and the sunlight-readable crew view
Hardware (if built)	Site Beacon buzzing at the trailer, its countdown still running after the connection drops

Demo rules: use the simulate buttons, clearly labeled, so nothing depends on live weather. Keep a recorded backup video. Never claim the system guarantees safety; it supports supervisors' decisions.

Sperry questions, Devpost checklist, and rules

Follow-up questions for the Sperry reps

The first four are answered (see "What Sperry's reps told us" near the top). Ask these next:

1. "You said it breaks down during something — was that hardware, handoffs, or something else?"
2. "With poor signal, how should a stop-work alert reach the crew: SMS, radio, or a device at the trailer?"
3. "Does satellite lightning flash data count as strikes, and is a labeled simulated strike fine for the demo?"
4. "How will you judge the 0–10 hazard index?"
5. "Must the 6 PM briefing actually be delivered, or is generating it enough?"
6. "Would a verifiable, tamper-evident safety record be valuable for your projects?"

Go back around hour 20 with the working demo and ask, "Is this heading in the right direction?"

Devpost checklist

- [ ] Public GitHub repo with README (required for AI/ML)
- [ ] Live URL on our domain, with test logins for supervisor and crew that expose no secrets (required for DeepSpace)
- [ ] Architecture diagram
- [ ] Demo video
- [ ] Submitted to all 13 challenges

For every sponsor challenge:

- [ ] Exact APIs and features used
- [ ] Where judges can see each one in the app
- [ ] Screenshot or GIF of the integration
- [ ] Why it matters for crew safety

Rules

- Every feature gets a simple working version before anything is polished.
- Rules decide alerts, the model decides scores, Gemini only explains; AI never starts or ends a stop-work.
- No personal data, photos, or locations on Solana; only hashes, counts, and timestamps.
- Show source timestamps everywhere; never call stale data live.
- Feature freeze at hour 30.
- Present CypressWatch as decision support, not a guarantee of safety.

Ready to submit when

- [ ] 50+ real sites ranked by a 0–10 score from a trained model
- [ ] Lightning countdown restarts on each strike and syncs to every device
- [ ] Crew app keeps the countdown, briefing, and map in airplane mode and uploads queued reports afterwards
- [ ] Flood rule fires after a heavy-rain night
- [ ] 6 PM briefing generates per time zone, in English and Spanish
- [ ] Crew photo reports are triaged, pinned, shared, and stored
- [ ] Solana Verify shows a match and catches a tampered record
- [ ] Auth0 roles enforced on the server, with step-up MFA on overrides

- [ ] Deployed on Vultr at our domain
- [ ] The team can run the core demo in 90 seconds
- [ ] Site Beacon works (only if built)

Hardware: Site Beacon (last priority)

Build this only after the hour-16 checkpoint confirms the core works. It answers the reps' poor-signal point: one beacon at the trailer, on the site's single connection, warns the whole crew out loud, including people whose phones have no signal or are in the truck.

How it works

- A laptop (or hotspot plus laptop) at the trailer runs a small Python script that receives stop-work and all-clear events and sends them to the board over USB serial.
- On a strike, the buzzer sounds, the LED turns red, and the LCD shows the countdown.
- The countdown runs on the board itself, so it keeps counting and shows the all-clear time even if the bridge script loses its connection; a new strike resets it when the connection returns.
- When the countdown ends, the LED turns green and the LCD shows "All clear".

Using our own Arduino

Yes, our own Arduino works for this, and nothing about it is permanent. Uploading a sketch only replaces the program on the board; afterwards, upload any other sketch (or the blank "BareMinimum" example from the Arduino IDE) and it's back to normal. The Grove modules unplug without soldering, so the board is returned exactly as it was.

Parts to collect from the MLH hardware lab

Must have

Item	Qty	Purpose
Our own Arduino (or a Raspberry Pi Pico from the lab)	1	Runs the countdown and controls the parts
Grove Buzzer	1	Stop-work alarm
Grove LED Socket Kit	1	Red = stop work, green = all clear

Item	Qty	Purpose
Grove LCD Screen	1	Countdown and all-clear time
Grove Cables	3–4	Plug-and-play connections
USB cable + laptop	1	Power and the connection to our backend

Ask the lab for: a Grove Base Shield (or Grove adapter) for our board. It isn't on the lab list, but Grove modules need it to plug into an Arduino. If there's none, take M/F jumper wires and a breadboard instead.

Optional, if time allows

Item	Purpose
Grove Button	Crew lead acknowledges the alert on the beacon
Grove Temperature and Barometer Sensor	On-site readings sent to Tiger Data next to the forecast
Grove Light Sensor	Adjusts LCD brightness for sun or night

Hardware checklist

- [ ] Install the Arduino IDE before the event
- [ ] Collect parts early; lab stock is shared
- [ ] Test the buzzer, LED, and LCD one at a time before combining them
- [ ] Serial bridge script receives a simulated strike
- [ ] Countdown keeps running after the bridge script stops
- [ ] Multimeter and tools stay at the lab table; bring the board there to debug
- [ ] Re-upload a blank sketch to our Arduino after the event

Estimated build time: about 3–5 hours with Grove parts.